

Sistema de Gestão de Territórios e Publicações (GTP)

Documento 09 – Docker

Projeto: Sistema de Gestão de Territórios e Publicações (GTP)

Versão: 1.0.0

Status: Em Elaboração

Data: Julho/2026

Autor: Fabio André Zatta

1 – Introdução

1.1 Objetivo

Este documento define a arquitetura de containerização do **Gestor de Territórios e Publicações (GTP)** utilizando a plataforma **Docker**.

Seu objetivo é estabelecer padrões para construção, empacotamento, distribuição, execução e manutenção dos serviços que compõem a aplicação, garantindo ambientes consistentes durante todas as fases do ciclo de vida do software.

A adoção do Docker proporciona isolamento entre serviços, padronização dos ambientes de desenvolvimento e produção, simplificação do processo de implantação e maior escalabilidade da solução.

1.2 Escopo

Este documento abrange:

- arquitetura dos contêineres;
- organização das imagens Docker;
- Dockerfiles;
- Docker Compose;
- redes Docker;
- volumes persistentes;
- variáveis de ambiente;
- gerenciamento de segredos;
- banco de dados PostgreSQL;
- execução do Flyway;
- integração entre backend e frontend;
- ambientes de desenvolvimento;
- homologação;
- produção;
- monitoramento;
- boas práticas de segurança.

1.3 Objetivos da Containerização

A utilização do Docker no GTP possui os seguintes objetivos:

- padronizar ambientes;
- eliminar diferenças entre máquinas de desenvolvimento;
- facilitar a implantação;
- reduzir problemas de configuração;
- aumentar a portabilidade;
- simplificar atualizações;
- permitir escalabilidade futura;
- facilitar integração contínua (CI);
- facilitar entrega contínua (CD).

1.4 Tecnologias Utilizadas

Tecnologia	Finalidade
Docker Engine	Execução dos contêineres
Docker Compose	Orquestração local dos serviços
Dockerfile	Construção das imagens
PostgreSQL 17	Banco de dados
Flyway	Migração do banco
Java 21	Backend
Spring Boot 3	API REST
React 19	Frontend
Vite	Build do frontend
Nginx	Servidor Web para o frontend
Alpine Linux	Imagens leves quando compatíveis
Eclipse Temurin	Runtime Java

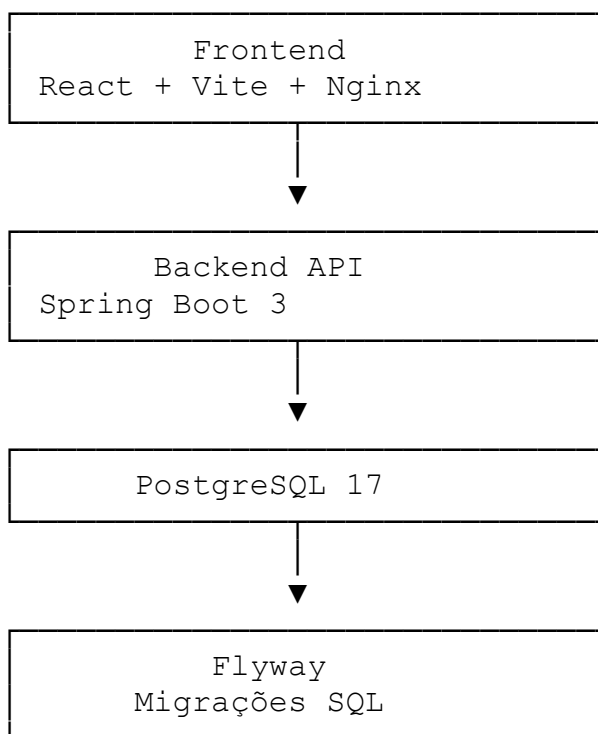
1.5 Benefícios Esperados

A adoção do Docker proporcionará:

- reprodutibilidade dos ambientes;
- facilidade de instalação;
- isolamento entre componentes;
- menor tempo de configuração;
- facilidade para novos desenvolvedores;
- melhor utilização dos recursos computacionais;
- maior confiabilidade nas implantações.

1.6 Componentes Containerizados

A arquitetura do GTP será composta pelos seguintes serviços:



Cada serviço executará em seu próprio contêiner, comunicando-se por meio de uma rede Docker dedicada.

1.7 Organização do Documento

Este documento está organizado nos seguintes capítulos:

Capítulo	Conteúdo
1	Introdução
2	Arquitetura Geral dos Contêineres
3	Dockerfiles
4	Docker Compose
5	Redes Docker
6	Volumes Persistentes
7	Banco de Dados e Flyway
8	Variáveis de Ambiente e Secrets
9	Build, Deploy e Publicação
10	Monitoramento, Logs e Boas Práticas
	Conclusão Diretrizes Finais

1.8 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral que será containerizada.
Documento 06 – Arquitetura Backend	Especifica os componentes do backend executados em contêineres.
Documento 07 – Arquitetura Frontend	Define a aplicação React executada em contêiner.
Documento 08 – Segurança	Estabelece diretrizes para proteção das imagens, redes e segredos.
Documento 10 – API REST	Define os serviços expostos pelo backend implantado via Docker.

1.9 Conclusão do Capítulo

A utilização do Docker no GTP representa um elemento fundamental da arquitetura da plataforma, permitindo a padronização dos ambientes, a simplificação das implantações e a escalabilidade da solução. A separação dos componentes em contêineres independentes favorece a manutenção, a evolução tecnológica e a adoção de práticas modernas de DevOps e integração contínua.

2 – Arquitetura Geral dos Contêineres

2.1 Objetivo

Este capítulo define a arquitetura geral dos contêineres utilizados pelo **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é estabelecer a organização dos serviços, os padrões de comunicação entre componentes, o isolamento das aplicações e as diretrizes para implantação em diferentes ambientes.

A arquitetura foi concebida para garantir portabilidade, escalabilidade, facilidade de manutenção e consistência entre desenvolvimento, homologação e produção.

2.2 Princípios da Containerização

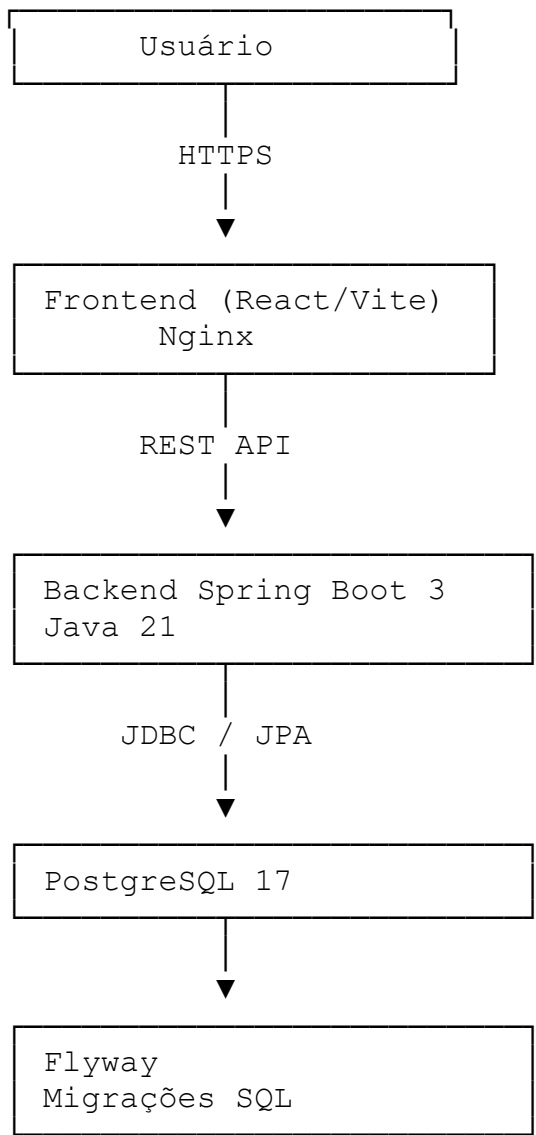
A estratégia de containerização seguirá os seguintes princípios:

- isolamento entre serviços;
- responsabilidade única por contêiner;
- imutabilidade das imagens;
- configuração por variáveis de ambiente;
- independência entre aplicações;
- comunicação por redes privadas;
- persistência desacoplada dos contêineres;
- reprodutibilidade dos ambientes.

Esses princípios simplificam a administração da infraestrutura e reduzem riscos de inconsistência.

2.3 Arquitetura Geral

A solução será composta por múltiplos contêineres especializados.



Cada serviço executará em um contêiner independente, conectado por uma rede Docker privada.

2.4 Organização dos Serviços

A arquitetura inicial será composta pelos seguintes serviços.

Serviço	Responsabilidade
Frontend	Interface do usuário construída com React e Vite.
Nginx	Servidor HTTP para entrega dos arquivos estáticos do frontend.

Serviço	Responsabilidade
---------	------------------

Backend	API REST desenvolvida com Spring Boot 3.
---------	--

PostgreSQL	Persistência dos dados da aplicação.
------------	--------------------------------------

Flyway	Controle de versão e migração do banco de dados.
--------	--

A separação favorece manutenção, escalabilidade e substituição independente dos componentes.

2.5 Responsabilidade dos Contêineres

Cada contêiner deverá possuir uma única responsabilidade principal.

Contêiner	Responsabilidade
-----------	------------------

Frontend	Servir a interface web.
----------	-------------------------

Backend	Executar a lógica de negócio e expor a API REST.
---------	--

PostgreSQL	Armazenar dados persistentes.
------------	-------------------------------

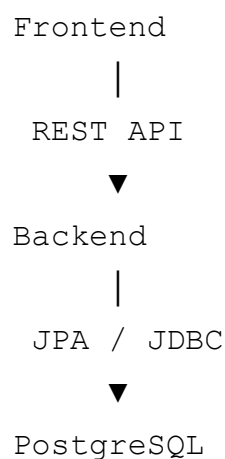
Flyway	Executar migrações do banco de dados.
--------	---------------------------------------

Essa abordagem está alinhada ao princípio de responsabilidade única.

2.6 Comunicação entre Serviços

A comunicação ocorrerá exclusivamente por meio da rede interna do Docker.

Fluxo principal:



O banco de dados não será exposto diretamente aos usuários.

2.7 Rede Docker

Os serviços compartilharão uma rede privada dedicada.

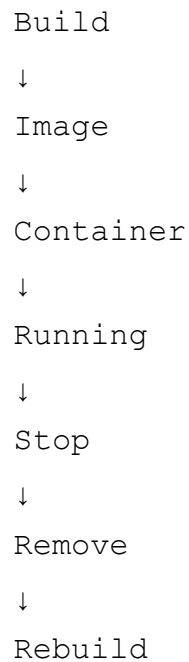
Características:

- comunicação isolada;
- resolução automática de nomes entre contêineres;
- ausência de exposição desnecessária de portas internas;
- facilidade de expansão para novos serviços.

Essa arquitetura reduz a superfície de ataque e simplifica a configuração.

2.8 Ciclo de Vida dos Contêineres

Cada serviço seguirá o ciclo de vida padrão do Docker.



A recriação dos contêineres não deverá implicar perda de dados persistentes.

2.9 Fluxo de Inicialização

A sequência recomendada para inicialização será:

1. criação da rede Docker;
2. inicialização do PostgreSQL;
3. verificação da disponibilidade do banco;
4. execução das migrações Flyway;

5. inicialização do Backend;
6. inicialização do Frontend.

Esse fluxo reduz falhas decorrentes de dependências entre serviços.

2.10 Ambientes de Execução

A arquitetura contemplará três ambientes principais.

Ambiente	Objetivo
----------	----------

Desenvolvimento	Uso pelos desenvolvedores.
-----------------	----------------------------

Homologação	Validação antes da produção.
-------------	------------------------------

Produção	Ambiente utilizado pelos usuários finais.
----------	---

Cada ambiente poderá possuir configurações específicas de infraestrutura.

2.11 Arquitetura do Ambiente de Desenvolvimento

No ambiente de desenvolvimento:

- código-fonte disponível localmente;
- hot reload para frontend;
- reinicialização automática do backend quando configurada;
- banco PostgreSQL em contêiner;
- Flyway executado automaticamente.

Esse ambiente prioriza produtividade.

2.12 Arquitetura de Homologação

A homologação utilizará imagens próximas às de produção.

Objetivos:

- validar integrações;
- executar testes finais;
- verificar migrações;
- identificar problemas antes da publicação.

2.13 Arquitetura de Produção

Em produção:

- imagens versionadas;
- configuração por variáveis de ambiente;
- HTTPS obrigatório;
- logs centralizados;
- backups periódicos;
- monitoramento contínuo;
- acesso restrito aos serviços internos.

2.14 Escalabilidade

A arquitetura foi preparada para crescimento futuro.

Possibilidades:

- múltiplas instâncias do backend;
- balanceamento de carga;
- replicação do banco de dados;
- cache distribuído;
- orquestração por Kubernetes.

Essas evoluções poderão ser implementadas sem alterações significativas na organização dos serviços.

2.15 Isolamento dos Serviços

Cada contêiner executará de forma independente.

Benefícios:

- redução de interferências;
- atualização individual;
- facilidade para testes;
- menor impacto em falhas isoladas.

2.16 Persistência

Os dados persistentes serão armazenados em volumes Docker.

Persistirão entre reinicializações:

- banco PostgreSQL;
- arquivos de configuração persistentes (quando aplicável);
- diretórios de backup.

As imagens permanecerão imutáveis.

2.17 Observabilidade

A arquitetura permitirá integração com mecanismos de monitoramento.

Exemplos:

- logs dos contêineres;
- métricas de utilização;
- consumo de memória;
- utilização de CPU;
- disponibilidade dos serviços.

2.18 Segurança da Arquitetura

As seguintes diretrizes deverão ser observadas:

- utilizar imagens oficiais ou homologadas;
- manter imagens atualizadas;
- limitar privilégios dos contêineres;
- utilizar redes privadas;
- proteger variáveis sensíveis;
- restringir portas expostas;
- realizar varreduras de vulnerabilidades nas imagens.

Essas práticas complementam as diretrizes estabelecidas no Documento 08 – Segurança.

2.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura lógica implementada pelos contêineres.
Documento 06 – Arquitetura Backend	Especifica o serviço backend executado em contêiner.
Documento 07 – Arquitetura Frontend	Define a estrutura da aplicação React entregue pelo contêiner frontend.
Documento 08 – Segurança	Estabelece os requisitos de segurança aplicados aos ambientes Docker.
Documento 10 – API REST	Define os serviços expostos pelo backend containerizado.

2.20 - Conclusão do Capítulo

A arquitetura geral dos contêineres do GTP estabelece uma estrutura modular, isolada e preparada para evolução. A separação dos serviços em contêineres independentes, a comunicação por redes privadas, a persistência desacoplada e a padronização dos ambientes proporcionam maior confiabilidade, facilidade de implantação e escalabilidade para a plataforma. Essa organização também cria uma base sólida para futuras integrações com soluções de orquestração, monitoramento e alta disponibilidade.

3 – Dockerfiles e Construção das Imagens

3.1 Objetivo

Este capítulo estabelece os padrões para criação dos **Dockerfiles** e construção das imagens do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é garantir que todas as imagens produzidas sejam:

- reproduzíveis;
- seguras;
- leves;
- versionadas;
- fáceis de manter;

- compatíveis com os diferentes ambientes da aplicação.

As imagens deverão atender aos requisitos definidos pelos documentos de Arquitetura e Segurança do projeto.

3.2 Princípios para Construção das Imagens

A construção das imagens seguirá os seguintes princípios:

- uma imagem para cada serviço;
- imagens imutáveis;
- utilização de **Multi-Stage Build**;
- redução do tamanho final;
- menor superfície de ataque;
- reutilização de cache durante o build;
- configuração externa por variáveis de ambiente;
- versionamento consistente.

3.3 Estrutura Geral dos Dockerfiles

Cada serviço possuirá seu próprio Dockerfile.

Estrutura sugerida:

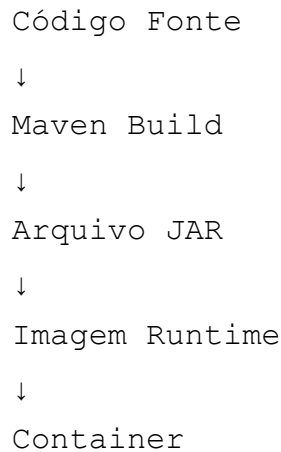
```
gtp/
├── backend/
│   └── Dockerfile
├── frontend/
│   └── Dockerfile
├── docker-compose.yml
└── .dockerignore
```

Essa organização facilita a manutenção e a independência entre os componentes.

3.4 Dockerfile do Backend

O backend será construído utilizando **Java 21**, **Spring Boot 3** e **Maven**.

Arquitetura de build:



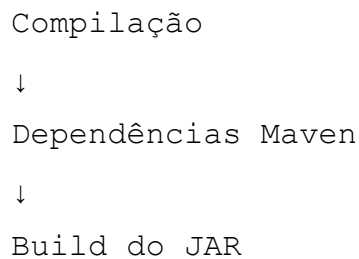
A separação entre as fases de compilação e execução reduz significativamente o tamanho da imagem final.

3.5 Multi-Stage Build

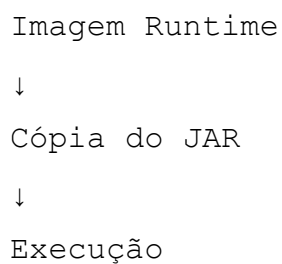
O backend utilizará **Multi-Stage Build**.

Fluxo:

Stage 1



Stage 2



Benefícios:

- imagens menores;
- menor consumo de armazenamento;
- menor tempo de download;
- redução de componentes desnecessários.

3.6 Imagem Base do Backend

Será utilizada uma imagem baseada em **Eclipse Temurin JDK 21** para compilação e **Eclipse Temurin JRE 21** (ou equivalente compatível) para execução.

Critérios para escolha:

- suporte oficial;
- compatibilidade com Spring Boot 3;
- atualizações frequentes;
- estabilidade;
- segurança.

3.7 Dockerfile do Frontend

O frontend será construído utilizando **Node.js** durante a compilação e **Nginx** para servir os arquivos estáticos.

Fluxo:

```
Código React
↓
npm install
↓
Build Vite
↓
Arquivos Estáticos
↓
Nginx
↓
Container
```

Essa arquitetura separa claramente o ambiente de build do ambiente de execução.

3.8 Multi-Stage no Frontend

O frontend também utilizará **Multi-Stage Build**.

Etapas:

1. instalação das dependências;
2. compilação da aplicação React;
3. geração dos arquivos estáticos;
4. cópia apenas dos arquivos finais para o Nginx.

Essa estratégia elimina dependências de desenvolvimento da imagem final.

3.9 Imagem Base do Frontend

A construção utilizará:

Etapas Imagem

Build Node.js LTS

Runtime Nginx Alpine

A utilização do **Nginx Alpine** reduz o tamanho da imagem e melhora o desempenho na entrega de conteúdo estático.

3.10 Arquivo .dockerignore

Cada serviço deverá possuir um arquivo .dockerignore.

Objetivos:

- reduzir contexto do build;
- acelerar construção;
- impedir inclusão de arquivos desnecessários;
- proteger informações locais.

Exemplos de exclusões:

- .git;
- node_modules;
- target;
- logs;
- arquivos temporários;

- IDEs;
- arquivos de configuração locais.

3.11 Otimização do Cache

Os Dockerfiles deverão ser organizados para maximizar o reaproveitamento do cache.

Boas práticas:

- copiar primeiro arquivos de dependências;
- instalar dependências antes de copiar o código-fonte;
- minimizar alterações em camadas superiores;
- evitar reconstruções desnecessárias.

Essa organização reduz significativamente o tempo de build.

3.12 Versionamento das Imagens

As imagens deverão seguir um padrão consistente de versionamento.

Exemplo:

```
gtp-backend:1.0.0
gtp-backend:1.1.0
gtp-backend:latest
```

```
gtp-frontend:1.0.0
gtp-frontend:1.1.0
gtp-frontend:latest
```

A utilização de versões explícitas facilita rastreabilidade e rollback.

3.13 Segurança das Imagens

As imagens deverão seguir práticas de hardening.

Diretrizes:

- utilizar imagens oficiais;
- manter imagens atualizadas;
- remover ferramentas desnecessárias;
- evitar execução como usuário root;

- limitar permissões;
- minimizar pacotes instalados.

Essas práticas reduzem a superfície de ataque.

3.14 Usuários Não Privilegiados

Sempre que possível, os contêineres deverão executar utilizando usuários não privilegiados.

Benefícios:

- redução de riscos;
- menor impacto de vulnerabilidades;
- aderência às boas práticas de segurança.

3.15 Variáveis de Ambiente

As imagens não deverão conter informações específicas de ambiente.

Configurações deverão ser fornecidas externamente.

Exemplos:

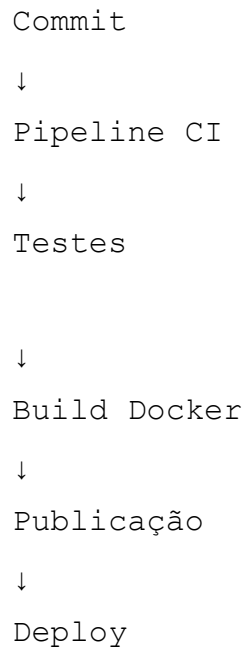
- URL da API;
- credenciais do banco;
- chaves JWT;
- perfis Spring;
- configurações de logs.

Essa estratégia aumenta a reutilização das imagens.

3.16 Build Automatizado

A construção das imagens poderá ser integrada a pipelines de CI/CD.

Fluxo sugerido:



Esse processo reduz falhas manuais e aumenta a confiabilidade das implantações.

3.17 Publicação em Registry

As imagens poderão ser publicadas em registries privados ou públicos.

Exemplos:

- Docker Hub;
- GitHub Container Registry (GHCR);
- GitLab Container Registry;
- Amazon Elastic Container Registry (ECR);
- Azure Container Registry (ACR).

A escolha dependerá da infraestrutura adotada pelo projeto.

3.18 Boas Práticas

Durante a criação dos Dockerfiles deverão ser observadas as seguintes diretrizes:

- utilizar Multi-Stage Build;
- reduzir o número de camadas;

- evitar arquivos desnecessários;
- utilizar imagens oficiais;
- executar processos como usuário não privilegiado;
- configurar variáveis externamente;
- manter dependências atualizadas;
- documentar alterações relevantes.

3.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define os componentes que serão empacotados nas imagens Docker.
Documento 06 – Arquitetura Backend	Especifica a estrutura da aplicação Spring Boot utilizada no Dockerfile do backend.
Documento 07 – Arquitetura Frontend	Define a aplicação React compilada pelo Dockerfile do frontend.
Documento 08 – Segurança	Estabelece diretrizes de hardening, proteção de imagens e gerenciamento de segredos.

3.20 Conclusão do Capítulo

A padronização dos Dockerfiles e do processo de construção das imagens garante que o GTP possa ser implantado de forma consistente em qualquer ambiente compatível com Docker. A adoção de **Multi-Stage Build**, imagens oficiais, usuários não privilegiados, cache otimizado e versionamento estruturado resulta em imagens menores, mais seguras e de fácil manutenção, contribuindo para a eficiência operacional e para a evolução contínua da plataforma.

4 – Docker Compose e Orquestração dos Serviços

4.1 Objetivo

Este capítulo estabelece a arquitetura de orquestração dos serviços do **Gestor de Territórios e Publicações (GTP)** utilizando **Docker Compose**.

Seu objetivo é padronizar a inicialização, comunicação e gerenciamento dos contêineres da aplicação, proporcionando ambientes consistentes, reproduzíveis e de fácil manutenção.

O Docker Compose será utilizado principalmente nos ambientes de desenvolvimento e homologação, mantendo uma estrutura compatível com futuras soluções de orquestração, como Kubernetes.

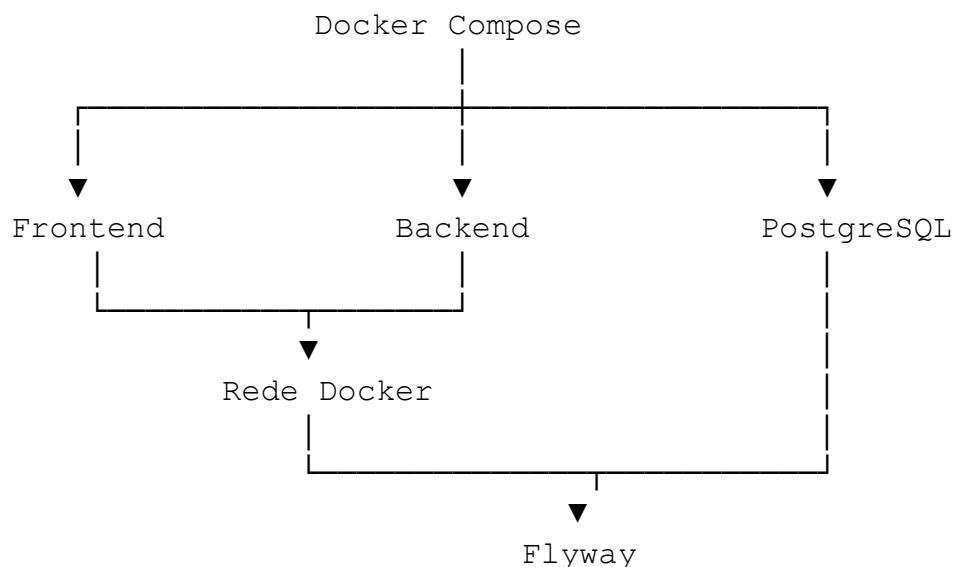
4.2 Papel do Docker Compose

O Docker Compose será responsável por:

- inicializar todos os serviços do sistema;
- criar automaticamente as redes Docker;
- configurar volumes persistentes;
- definir variáveis de ambiente;
- estabelecer dependências entre os contêineres;
- facilitar a execução do ambiente completo com um único comando.

4.3 Arquitetura Geral da Orquestração

A composição dos serviços seguirá a arquitetura abaixo.



Todos os serviços compartilharão uma rede privada criada automaticamente pelo Docker Compose.

4.4 Estrutura do Arquivo docker-compose.yml

A organização lógica do arquivo será composta pelos seguintes blocos:

- definição da versão do Compose;
- serviços;
- redes;
- volumes;
- variáveis de ambiente;
- dependências;
- políticas de reinicialização;
- verificações de integridade (*health checks*).

Essa organização favorece a legibilidade e a manutenção.

4.5 Serviços Definidos

O ambiente inicial do GTP será composto pelos seguintes serviços.

Serviço Finalidade

frontend Interface React servida pelo Nginx.

backend API REST desenvolvida com Spring Boot 3.

postgres Banco de dados PostgreSQL 17.

flyway Execução automática das migrações do banco de dados.

Cada serviço possuirá configuração própria, mantendo independência operacional.

4.6 Dependências entre Serviços

A ordem lógica de inicialização será:

PostgreSQL

↓

Flyway

↓

Backend

↓

Frontend

O uso da diretiva `depends_on` deverá ser complementado por verificações de integridade, garantindo que um serviço esteja realmente disponível antes da inicialização do próximo.

4.7 Redes Docker

Será criada uma rede dedicada para a comunicação entre os serviços.

Características:

- isolamento da infraestrutura;
- resolução automática de nomes;
- comunicação interna segura;
- possibilidade de expansão para novos serviços.

O banco de dados permanecerá acessível apenas pela rede interna.

4.8 Volumes Persistentes

Os dados persistentes serão armazenados em volumes Docker.

Exemplos de utilização:

Volume	Finalidade
postgres-data	Persistência do banco PostgreSQL.
backups	Armazenamento de cópias de segurança (quando aplicável).
logs	Persistência de registros da aplicação, conforme estratégia de implantação.

A utilização de volumes garante que os dados sejam preservados mesmo após a recriação dos contêineres.

4.9 Variáveis de Ambiente

As configurações específicas de cada ambiente serão fornecidas externamente.

Exemplos:

- perfil do Spring Boot;
- URL da API;
- credenciais do banco;

- porta da aplicação;
- chaves JWT;
- configurações de logs.

Essa abordagem permite reutilizar a mesma imagem Docker em diferentes ambientes.

4.10 Arquivos de Configuração

As configurações poderão ser organizadas por ambiente.

Exemplo:

```
.env.development
.env.test
.env.staging
.env.production
```

Cada arquivo conterá apenas parâmetros específicos do respectivo ambiente.

4.11 Health Checks

Os serviços críticos deverão possuir verificações de integridade.

Objetivos:

- confirmar disponibilidade do serviço;
- identificar falhas rapidamente;
- impedir inicialização prematura de componentes dependentes;
- aumentar a confiabilidade do ambiente.

Exemplos de verificações:

- PostgreSQL respondendo a conexões;
- Backend respondendo ao endpoint de *health*;
- Frontend disponível via servidor HTTP.

4.12 Políticas de Reinicialização

Os serviços utilizarão políticas de reinicialização apropriadas.

Possíveis estratégias:

Política	Utilização
no	Não reiniciar automaticamente.
on-failure	Reiniciar apenas após falhas.
unless-stopped	Reiniciar automaticamente, exceto quando interrompido manualmente.
always	Reiniciar sempre que possível.

A escolha dependerá do ambiente de execução.

4.13 Perfis de Execução

O Docker Compose poderá utilizar perfis para ativar serviços específicos.

Exemplos:

Perfil	Serviços
development	Backend, Frontend, PostgreSQL, Flyway.
homologation	Configuração semelhante à produção para validação.
production	Serviços essenciais e configurações otimizadas para operação.

Essa organização simplifica a administração dos ambientes.

4.14 Fluxo de Inicialização

O fluxo recomendado para criação do ambiente será:

```

Docker Compose Up
↓
Criação da Rede
↓
Inicialização do PostgreSQL
↓
Health Check
↓
Execução do Flyway
↓
Inicialização do Backend
↓
Health Check
↓
Inicialização do Frontend
↓
Ambiente Disponível

```

4.15 Escalabilidade

Embora o Docker Compose seja voltado principalmente para ambientes locais, sua estrutura deverá facilitar a evolução para soluções mais robustas.

A arquitetura permitirá futuramente:

- múltiplas instâncias do backend;
- balanceamento de carga;
- serviços adicionais;
- cache distribuído;
- mensageria;
- migração para Kubernetes.

4.16 Monitoramento

O ambiente deverá permitir integração com mecanismos de monitoramento.

Informações relevantes:

- utilização de CPU;
- consumo de memória;
- disponibilidade dos serviços;
- tempo de resposta;
- utilização de armazenamento;
- eventos dos contêineres.

4.17 Segurança da Orquestração

As seguintes diretrizes deverão ser observadas:

- utilizar redes privadas;
- limitar portas expostas ao host;
- armazenar segredos fora do arquivo Compose;
- utilizar imagens confiáveis;
- manter dependências atualizadas;
- restringir privilégios dos contêineres;
- utilizar usuários não privilegiados sempre que possível.

4.18 Boas Práticas

Durante a utilização do Docker Compose deverão ser seguidas as seguintes recomendações:

- manter os serviços independentes;
- evitar dependências desnecessárias;
- utilizar nomes padronizados;
- documentar variáveis de ambiente;
- preservar volumes persistentes;
- realizar testes de inicialização periódicos;
- revisar configurações de rede e segurança.

4.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define os componentes que serão orquestrados pelo Docker Compose.
Documento 06 – Arquitetura Backend	Especifica o serviço backend executado na composição Docker.
Documento 07 – Arquitetura Frontend	Define o serviço frontend incluído na composição.
Documento 08 – Segurança	Estabelece requisitos de segurança para redes, variáveis de ambiente e gerenciamento de segredos.

4.20 Conclusão do Capítulo

O Docker Compose fornece uma forma padronizada de orquestrar todos os serviços do GTP, permitindo a criação de ambientes completos com baixo esforço operacional. A utilização de redes privadas, volumes persistentes, verificações de integridade, perfis de execução e políticas de reinicialização contribui para um ambiente consistente, seguro e preparado para evoluir para soluções de orquestração mais avançadas quando necessário.

5 – Redes Docker e Comunicação entre Serviços

5.1 Objetivo

Este capítulo estabelece a arquitetura de redes Docker utilizada pelo **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é definir como os serviços da aplicação se comunicam entre si, garantindo isolamento, segurança, desempenho e facilidade de administração.

A comunicação entre os componentes deverá ocorrer exclusivamente por redes controladas, reduzindo a exposição da infraestrutura e assegurando que apenas os serviços autorizados possam trocar informações.

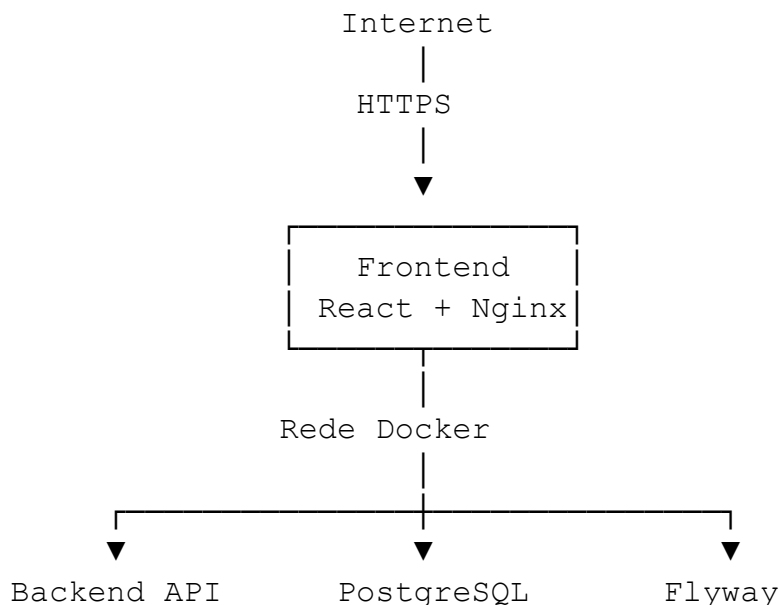
5.2 Princípios da Arquitetura de Rede

A infraestrutura de rede seguirá os seguintes princípios:

- isolamento entre serviços;
- comunicação interna privada;
- exposição mínima de portas;
- resolução automática de nomes;
- independência entre ambientes;
- segurança por padrão (*Secure by Default*);
- escalabilidade da infraestrutura;
- simplicidade de configuração.

5.3 Arquitetura Geral da Rede

A comunicação entre os componentes ocorrerá por meio de uma rede Docker dedicada.



Os contêineres comunicar-se-ão utilizando os nomes dos serviços definidos na composição Docker, sem necessidade de endereços IP fixos.

5.4 Comunicação Entre os Serviços

Cada componente possui responsabilidades específicas na comunicação da plataforma.

Origem	Destino	Finalidade
--------	---------	------------

Navegador Frontend		Carregamento da interface web.
--------------------	--	--------------------------------

Frontend	Backend	Consumo da API REST.
----------	---------	----------------------

Backend	PostgreSQL	Persistência e consulta de dados.
---------	------------	-----------------------------------

Flyway	PostgreSQL	Execução das migrações do banco de dados.
--------	------------	---

Não haverá comunicação direta entre o navegador e o banco de dados.

5.5 Resolução de Nomes

O Docker fornecerá resolução automática de nomes (*DNS* interno).

Exemplos de nomes de serviços:

Serviço	Nome de Rede
---------	--------------

Frontend	frontend
----------	----------

Backend	backend
---------	---------

PostgreSQL	postgres
------------	----------

Flyway	flyway
--------	--------

Essa funcionalidade elimina a necessidade de configuração manual de endereços IP.

5.6 Redes por Ambiente

Cada ambiente poderá utilizar redes independentes.

Ambiente	Objetivo
----------	----------

Desenvolvimento	Uso local pelos desenvolvedores.
-----------------	----------------------------------

Homologação	Testes integrados antes da produção.
-------------	--------------------------------------

Produção	Operação em ambiente definitivo.
----------	----------------------------------

A separação evita interferências entre ambientes e facilita o gerenciamento.

5.7 Isolamento dos Serviços

O isolamento da rede deverá impedir acessos indevidos entre componentes externos e internos.

Diretrizes:

- o banco de dados não será acessível diretamente pela Internet;
- apenas o backend poderá estabelecer conexões com o PostgreSQL;
- o Flyway comunicará exclusivamente com o banco de dados;
- o frontend será o único serviço exposto aos usuários finais.

Essa organização reduz significativamente a superfície de ataque.

5.8 Exposição de Portas

A publicação de portas deverá seguir o princípio do menor privilégio.

Serviço	Exposição
---------	-----------

Frontend	Porta HTTP/HTTPS pública.
----------	---------------------------

Serviço	Exposição
---------	-----------

Backend	Exposição restrita conforme ambiente de implantação.
---------	--

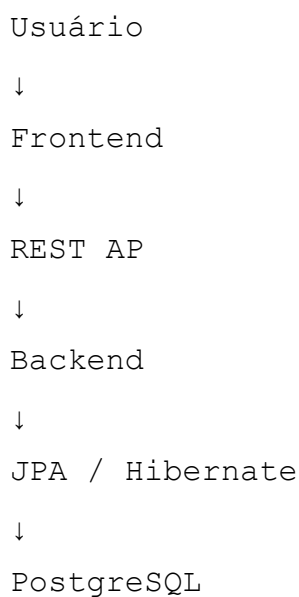
PostgreSQL	Acesso apenas interno, salvo necessidade administrativa controlada.
------------	---

Flyway	Sem exposição externa.
--------	------------------------

A configuração poderá variar conforme o ambiente de execução.

5.9 Fluxo de Comunicação

O fluxo principal de comunicação será:



Esse fluxo preserva a separação de responsabilidades entre interface, lógica de negócio e persistência.

5.10 Comunicação Segura

As seguintes diretrizes deverão ser observadas:

- utilização de HTTPS para acesso dos usuários;
- criptografia da comunicação sempre que aplicável;
- autenticação das requisições à API;
- validação de certificados em ambientes de produção;
- restrição do tráfego interno aos serviços autorizados.

5.11 Redes Privadas

A comunicação entre os serviços utilizará redes privadas Docker.

Benefícios:

- isolamento do tráfego;
- redução da exposição da infraestrutura;
- maior controle sobre os acessos;
- facilidade de expansão da arquitetura.

5.12 Integração com Proxy Reverso

A arquitetura está preparada para integração com um proxy reverso.

Possíveis responsabilidades do proxy:

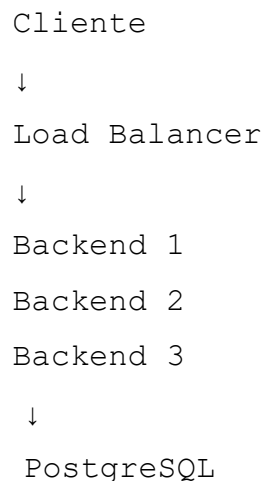
- terminação TLS;
- redirecionamento de requisições;
- balanceamento de carga;
- compressão de conteúdo;
- cache de recursos estáticos;
- controle de cabeçalhos HTTP.

A adoção do proxy dependerá da infraestrutura de implantação.

5.13 Preparação para Balanceamento de Carga

A arquitetura de rede permitirá a distribuição de requisições entre múltiplas instâncias do backend.

Fluxo futuro:



Essa estrutura facilita a escalabilidade horizontal da aplicação.

5.14 Comunicação com Serviços Externos

Quando necessário, o backend poderá estabelecer comunicação com serviços externos.

Exemplos:

- serviços de autenticação;
- provedores de e-mail;
- APIs governamentais;
- serviços de monitoramento;
- provedores de armazenamento.

Todas as integrações deverão utilizar conexões seguras e autenticação adequada.

5.15 Monitoramento da Rede

A infraestrutura deverá permitir acompanhamento de indicadores como:

- disponibilidade dos serviços;
- tempo de resposta;
- conexões estabelecidas;
- tráfego entre contêineres;
- falhas de comunicação;
- utilização de largura de banda.

Essas informações apoiarão a operação e a identificação de incidentes.

5.16 Segurança da Comunicação

A comunicação entre serviços deverá observar as seguintes diretrizes:

- limitar portas abertas;
- utilizar redes dedicadas;
- impedir acesso direto ao banco de dados;
- evitar exposição de serviços administrativos;
- proteger variáveis de ambiente;
- utilizar imagens atualizadas;

- aplicar políticas de firewall quando aplicável.

5.17 Boas Práticas

Durante a configuração das redes Docker deverão ser observadas as seguintes recomendações:

- utilizar nomes padronizados para serviços;
- evitar dependência de endereços IP fixos;
- documentar portas utilizadas;
- revisar periodicamente as regras de acesso;
- minimizar a exposição de portas ao host;
- separar ambientes de desenvolvimento, homologação e produção.

5.18 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura lógica implementada sobre a infraestrutura de rede.
Documento 06 – Arquitetura Backend	Especifica o serviço backend que utiliza a rede Docker para acesso ao banco de dados e demais serviços.
Documento 07 – Arquitetura Frontend	Define a comunicação da interface React com a API REST.
Documento 08 – Segurança	Estabelece as diretrizes de proteção das comunicações, autenticação e controle de acesso.
Documento 10 – API REST	Define os contratos de comunicação entre frontend e backend.

5.19 Conclusão do Capítulo

A arquitetura de redes Docker do GTP estabelece um ambiente seguro, organizado e preparado para evolução. A utilização de redes privadas, resolução automática de nomes, isolamento dos serviços, exposição controlada de portas e preparação para proxies reversos e balanceadores de carga garante uma infraestrutura consistente, reduz a superfície de ataque e facilita a escalabilidade da plataforma. Essas diretrizes asseguram que a comunicação entre os componentes ocorra de forma eficiente, confiável e alinhada às boas práticas modernas de containerização.

6 – Volumes Persistentes e Gerenciamento de Dados

6.1 Objetivo

Este capítulo estabelece as diretrizes para utilização de **volumes persistentes** e gerenciamento de dados no ambiente Docker do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é garantir que os dados da aplicação permaneçam íntegros, disponíveis e recuperáveis, mesmo após a atualização, reinicialização ou substituição dos contêineres.

A persistência será tratada como um componente essencial da arquitetura da infraestrutura, assegurando continuidade operacional e confiabilidade da plataforma.

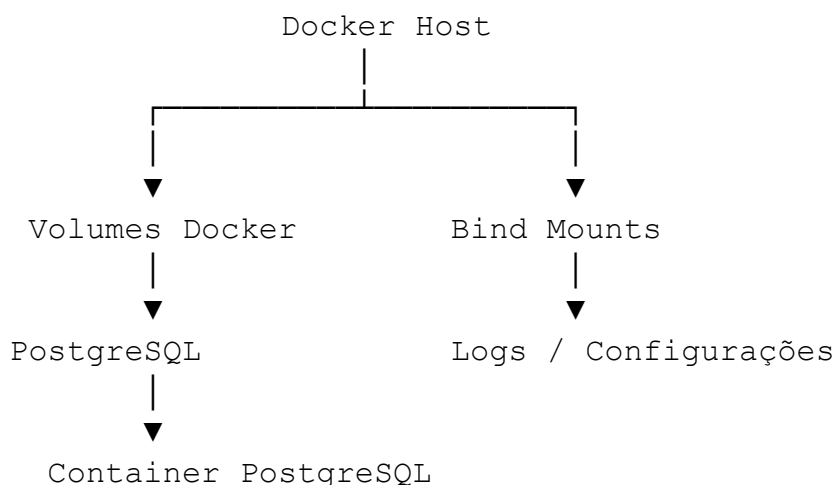
6.2 Princípios da Persistência

A estratégia de persistência seguirá os seguintes princípios:

- independência entre dados e contêineres;
- preservação das informações durante atualizações;
- isolamento dos dados por serviço;
- facilidade de backup e restauração;
- proteção contra perda acidental;
- escalabilidade da infraestrutura;
- segurança dos dados armazenados.

6.3 Arquitetura da Persistência

A persistência será baseada em volumes Docker independentes dos contêineres.



Essa arquitetura permite que os dados sobrevivam ao ciclo de vida dos contêineres.

6.4 Tipos de Armazenamento

Serão utilizados diferentes mecanismos de armazenamento conforme a finalidade.

Tipo	Finalidade
Volume Nomeado	Persistência do banco de dados e dados permanentes.
Bind Mount	Compartilhamento de arquivos durante o desenvolvimento.
Volume Temporário	Dados transitórios ou caches, quando aplicável.

Cada tipo será utilizado conforme as necessidades do ambiente.

6.5 Volumes do PostgreSQL

O banco de dados utilizará um volume dedicado.

Responsabilidades:

- armazenar tabelas;
- índices;
- sequências;
- transações;
- metadados;
- arquivos internos do PostgreSQL.

Esse volume será preservado durante atualizações da aplicação.

6.6 Persistência dos Dados da Aplicação

Todos os dados funcionais do GTP permanecerão no banco PostgreSQL.

Exemplos:

- usuários;
- congregações;
- territórios;
- publicações;
- históricos;
- auditoria;

- parametrizações.

Nenhuma informação permanente será armazenada no sistema de arquivos do contêiner do backend.

6.7 Armazenamento de Logs

A estratégia de armazenamento de logs dependerá do ambiente.

Em desenvolvimento:

- logs poderão permanecer disponíveis no console.

Em homologação e produção:

- poderá ser utilizado volume dedicado;
- integração com sistemas centralizados de observabilidade;
- retenção conforme política operacional.

6.8 Arquivos Temporários

Arquivos temporários gerados pela aplicação deverão possuir ciclo de vida controlado.

Exemplos:

- arquivos de importação;
- arquivos de exportação;
- documentos temporários;
- cache de processamento.

Sempre que possível, esses arquivos deverão ser removidos automaticamente após sua utilização.

6.9 Estratégia para Backups

A arquitetura deverá permitir backups completos e incrementais do banco de dados.

O processo poderá incluir:

1. criação do backup;
2. validação do arquivo gerado;
3. armazenamento seguro;
4. testes periódicos de restauração;
5. monitoramento da execução.

A política de frequência será definida conforme os requisitos operacionais.

6.10 Recuperação de Dados

A infraestrutura deverá permitir restauração rápida dos dados.

O processo de recuperação compreenderá:

- restauração do banco PostgreSQL;
- reaplicação das migrações, quando necessário;
- validação da integridade dos dados;
- reinicialização dos serviços.

Esses procedimentos deverão ser documentados e testados periodicamente.

6.11 Gerenciamento do Ciclo de Vida dos Volumes

Os volumes deverão possuir políticas definidas para:

- criação;
- utilização;
- manutenção;
- limpeza;
- arquivamento;
- remoção.

A exclusão de volumes contendo dados permanentes deverá ocorrer apenas mediante procedimentos controlados.

6.12 Migração de Dados

A arquitetura deverá permitir migração segura entre ambientes.

Exemplos:

- desenvolvimento → homologação;
- homologação → produção;
- migração entre servidores;
- atualização da infraestrutura.

Os procedimentos deverão preservar a integridade e a consistência dos dados.

6.13 Segurança dos Dados Persistidos

Os dados armazenados deverão ser protegidos.

Diretrizes:

- controle de acesso aos volumes;
- permissões restritas;
- backups protegidos;
- criptografia quando exigida pela política de segurança;
- monitoramento de acessos administrativos.

Essas medidas complementam as diretrizes definidas no Documento 08 – Segurança.

6.14 Estratégia para Desenvolvimento

Durante o desenvolvimento:

- utilização de volumes persistentes para o PostgreSQL;
- possibilidade de *bind mounts* para facilitar alterações de código;
- reinicialização frequente dos contêineres sem perda de dados.

Essa abordagem aumenta a produtividade da equipe.

6.15 Estratégia para Produção

Em produção:

- utilização exclusiva de volumes persistentes;
- backups automatizados;
- controle rigoroso de acesso;
- monitoramento contínuo;
- procedimentos documentados de recuperação.

Essas práticas aumentam a disponibilidade da aplicação.

6.16 Monitoramento dos Volumes

Os seguintes indicadores deverão ser acompanhados:

- espaço disponível;
- crescimento da base de dados;

- utilização dos volumes;
- falhas de leitura ou gravação;
- tempo de resposta do armazenamento.

O monitoramento contínuo auxilia na prevenção de indisponibilidades.

6.17 Limpeza de Recursos

A manutenção periódica deverá contemplar:

- remoção de volumes obsoletos;
- limpeza de caches;
- exclusão de arquivos temporários;
- revisão de backups antigos;
- reorganização do armazenamento.

Essas atividades deverão seguir procedimentos previamente definidos.

6.18 Boas Práticas

Durante a utilização de volumes persistentes deverão ser observadas as seguintes diretrizes:

- nunca armazenar dados permanentes no sistema de arquivos do contêiner;
- utilizar volumes nomeados para bancos de dados;
- proteger permissões dos diretórios;
- realizar backups periódicos;
- testar restaurações regularmente;
- documentar procedimentos de recuperação;
- monitorar crescimento do armazenamento;
- revisar periodicamente a política de retenção.

6.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define os componentes responsáveis pelo tratamento dos dados persistidos.

Documento	Relação
Documento 06 – Arquitetura Backend	Especifica a camada de persistência baseada em JPA/Hibernate e PostgreSQL.
Documento 08 – Segurança	Define as políticas de proteção, backup e criptografia dos dados armazenados.
Documento 10 – API REST	Utiliza os dados persistidos para atender às operações da aplicação.

6.20 Conclusão do Capítulo

A estratégia de persistência do GTP garante que os dados da plataforma permaneçam protegidos e independentes do ciclo de vida dos contêineres. A utilização de volumes Docker, políticas de backup, procedimentos de recuperação, controle de acesso e monitoramento contínuo assegura a integridade e a disponibilidade das informações, permitindo que a infraestrutura evolua sem comprometer a continuidade operacional da aplicação.

7 – Banco de Dados, Flyway e Inicialização Automatizada

7.1 Objetivo

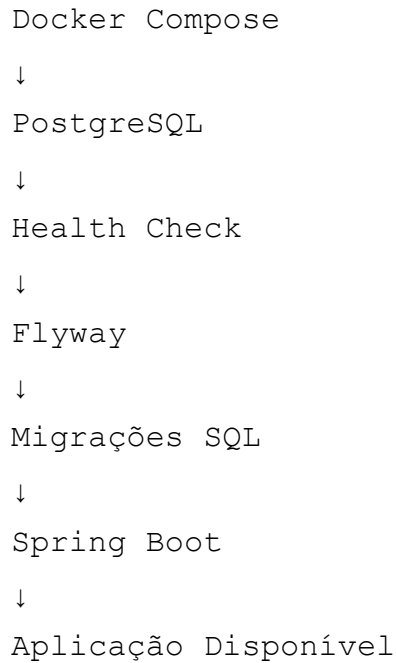
Este capítulo estabelece as diretrizes para utilização do banco de dados **PostgreSQL 17** e do **Flyway** na infraestrutura Docker do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é garantir que o esquema do banco de dados seja mantido sob controle de versão, permitindo evolução contínua, implantação automatizada e sincronização entre os ambientes de desenvolvimento, homologação e produção.

7.2 Arquitetura Geral

O banco de dados será executado em um contêiner dedicado, separado da aplicação.

O processo de inicialização seguirá a sequência abaixo.



Essa ordem garante que o backend seja iniciado apenas após a conclusão das migrações necessárias.

7.3 Serviço PostgreSQL

O PostgreSQL será responsável por armazenar todas as informações persistentes do GTP.

Entre os principais grupos de dados encontram-se:

- usuários;
- perfis e permissões;
- congregações;
- territórios;
- publicações;
- designações;
- auditoria;
- configurações do sistema;
- parâmetros operacionais.

O banco será executado em um contêiner independente, utilizando volumes persistentes para armazenamento dos dados.

7.4 Configuração do Banco de Dados

A configuração do PostgreSQL deverá ser realizada por meio de variáveis de ambiente.

Exemplos de parâmetros:

Parâmetro	Finalidade
Nome do banco	Identificação da base de dados.
Usuário	Conta de acesso da aplicação.
Senha	Credencial protegida do banco de dados.
Porta	Porta de comunicação do serviço.
Codificação	Configuração de caracteres (UTF-8).
Fuso horário	Padronização temporal da aplicação.

Essas configurações deverão variar conforme o ambiente de execução.

7.5 Gerenciamento de Conexões

O backend utilizará um pool de conexões para otimizar o acesso ao banco de dados.

Objetivos:

- reduzir tempo de resposta;
- reutilizar conexões existentes;
- controlar concorrência;
- evitar excesso de conexões simultâneas.

Os parâmetros do pool deverão ser ajustados conforme a capacidade da infraestrutura.

7.6 Flyway

O Flyway será responsável pelo controle de versão do esquema do banco de dados.

Suas principais responsabilidades incluem:

- execução automática de migrações;
- versionamento do esquema;
- rastreamento das alterações aplicadas;

- prevenção de inconsistências entre ambientes;
- suporte à evolução incremental do banco.

Todas as alterações estruturais deverão ser realizadas exclusivamente por migrações versionadas.

7.7 Estrutura das Migrações

As migrações seguirão um padrão de organização.

Exemplo:

```
db/  
└─ migration/  
    V1__Criacao_inicial.sql  
    V2__Usuarios.sql  
    V3__Congregacoes.sql  
    V4__Territorios.sql  
    V5__Publicacoes.sql  
    V6__Indices.sql
```

A nomenclatura deverá ser consistente, permitindo fácil identificação da finalidade de cada migração.

7.8 Processo de Inicialização

Durante a inicialização da aplicação, o seguinte fluxo deverá ser observado:

1. criação do contêiner PostgreSQL;
2. verificação da disponibilidade do banco;
3. execução das migrações pelo Flyway;
4. validação da versão do esquema;
5. inicialização do backend;
6. disponibilização da API.

Esse processo reduz riscos de incompatibilidade entre código e banco de dados.

7.9 Controle de Versão do Esquema

Cada alteração estrutural será registrada em uma nova migração.

Boas práticas:

- nunca alterar migrações já executadas em produção;
- criar novas versões para cada modificação;
- manter histórico completo das alterações;
- revisar as migrações antes da implantação.

Essa abordagem garante rastreabilidade e facilita auditorias.

7.10 Tratamento de Falhas nas Migrações

Caso ocorra erro durante uma migração:

- a inicialização do backend deverá ser interrompida;
- o erro deverá ser registrado em logs;
- nenhuma migração posterior deverá ser executada;
- a correção deverá ocorrer antes de uma nova tentativa de implantação.

Esse comportamento evita estados inconsistentes do banco de dados.

7.11 Dados Iniciais

Quando necessário, poderão ser carregados dados iniciais (*seed data*).

Exemplos:

- perfis padrão;
- permissões básicas;
- configurações iniciais;
- parâmetros globais.

Esses dados deverão ser versionados e controlados juntamente com as migrações.

7.12 Ambientes de Execução

O comportamento do Flyway poderá variar conforme o ambiente.

Ambiente	Estratégia
----------	------------

Desenvolvimento	Execução automática das migrações a cada inicialização.
-----------------	---

Ambiente	Estratégia
Homologação	Migrações controladas antes dos testes.
Produção	Migrações executadas conforme processo formal de implantação.

Essa diferenciação reduz riscos operacionais em ambientes críticos.

7.13 Backup Antes das Migrações

Antes da aplicação de migrações em produção, recomenda-se:

- realizar backup completo do banco;
- validar a integridade do backup;
- confirmar a possibilidade de restauração;
- registrar a versão do esquema anterior.

Essas medidas facilitam procedimentos de recuperação em caso de falhas.

7.14 Monitoramento

O banco de dados deverá ser monitorado continuamente.

Indicadores recomendados:

- disponibilidade;
- tempo de resposta;
- utilização de conexões;
- crescimento do banco;
- espaço em disco;
- execução das migrações;
- falhas de inicialização.

7.15 Segurança

As seguintes diretrizes deverão ser observadas:

- utilização de credenciais protegidas;
- acesso restrito ao banco de dados;
- armazenamento seguro das senhas;
- comunicação apenas pela rede interna do Docker;

- backups protegidos;
- atualização periódica da imagem do PostgreSQL.

7.16 Estratégia de Recuperação

Em caso de falha durante a implantação:

1. interromper a inicialização da aplicação;
2. analisar os logs do Flyway;
3. restaurar o backup, se necessário;
4. corrigir a migração;
5. repetir o processo de implantação.

A recuperação deverá preservar a integridade dos dados.

7.17 Boas Práticas

Durante o gerenciamento do banco de dados deverão ser observadas as seguintes recomendações:

- versionar todas as alterações estruturais;
- nunca modificar migrações já executadas em produção;
- manter scripts pequenos e objetivos;
- documentar mudanças relevantes;
- validar migrações em homologação antes da produção;
- automatizar a execução sempre que possível;
- monitorar continuamente o ambiente.

7.18 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define os componentes responsáveis pelo acesso aos dados persistidos.
Documento 06 – Arquitetura Backend	Especifica a camada de persistência baseada em JPA/Hibernate e PostgreSQL.
Documento 08 – Segurança	Estabelece diretrizes para proteção do banco de dados, credenciais e backups.

Documento	Relação
Documento 10 – API REST	Consome os dados persistidos para atender às operações da aplicação.

7.19 Conclusão do Capítulo

A utilização do PostgreSQL em conjunto com o Flyway proporciona ao GTP uma estratégia robusta de gerenciamento do banco de dados, baseada em versionamento, automação e rastreabilidade. A execução controlada das migrações, o monitoramento contínuo, as políticas de backup e recuperação e a integração com a infraestrutura Docker garantem consistência entre os ambientes e reduzem significativamente os riscos durante atualizações da aplicação.

8 – Variáveis de Ambiente, Configuração e Gerenciamento de Segredos

8.1 Objetivo

Este capítulo estabelece as diretrizes para gerenciamento das configurações da infraestrutura Docker do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é definir como parâmetros de configuração, credenciais, chaves criptográficas e demais informações sensíveis serão administradas de forma segura, permitindo que a mesma imagem Docker seja utilizada em diferentes ambientes sem necessidade de modificações no código-fonte.

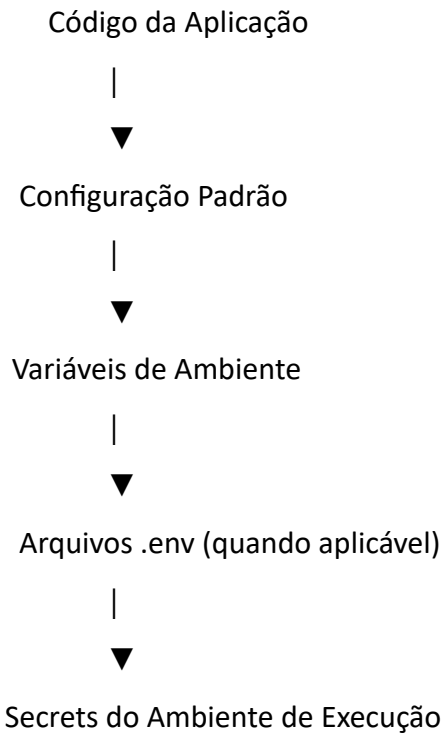
8.2 Princípios Gerais

A configuração da aplicação seguirá os seguintes princípios:

- separação entre código e configuração;
- configuração externa (*Externalized Configuration*);
- reutilização das imagens Docker;
- proteção de informações sensíveis;
- rastreabilidade das alterações;
- facilidade de manutenção;
- compatibilidade com processos de CI/CD;
- aderência às boas práticas de DevSecOps.

8.3 Arquitetura das Configurações

As configurações serão organizadas em camadas.



Essa arquitetura permite flexibilidade sem comprometer a segurança.

8.4 Categorias de Configuração

As configurações do GTP serão agrupadas conforme sua finalidade.

Categoria	Exemplos
Banco de Dados	Host, porta, nome da base, usuário.
Segurança	Chaves JWT, algoritmos, tempo de expiração.
Backend	Perfil Spring, portas, logs.
Frontend	URL da API, ambiente de execução.
Docker	Redes, volumes e parâmetros de inicialização.
Observabilidade	Configuração de logs, métricas e monitoramento.

Essa organização facilita a gestão e a documentação das configurações.

8.5 Variáveis de Ambiente

As variáveis de ambiente serão o principal mecanismo de configuração da aplicação.

Exemplos:

- perfil da aplicação;
- URL da API;
- endereço do banco de dados;
- usuário do banco;
- senha do banco;
- chave JWT;
- porta da aplicação;
- nível de log.

Essas variáveis poderão ser fornecidas pelo Docker Compose, pelo ambiente de hospedagem ou por ferramentas de orquestração.

8.6 Arquivos .env

Nos ambientes de desenvolvimento poderão ser utilizados arquivos .env para facilitar a configuração local.

Exemplo de organização:

```
.env.development
.env.homologation
.env.production
```

Esses arquivos não deverão conter informações sensíveis quando armazenados em repositórios públicos.

8.7 Perfis do Spring Boot

O backend utilizará perfis do Spring Boot para adequar seu comportamento a cada ambiente.

Perfil	Finalidade
--------	------------

development	Desenvolvimento local.
-------------	------------------------

homologation	Testes integrados e validação.
--------------	--------------------------------

production	Ambiente de produção.
------------	-----------------------

Cada perfil poderá possuir configurações específicas para banco de dados, logs, monitoramento e desempenho.

8.8 Configuração do Frontend

O frontend também utilizará variáveis de ambiente para parametrização.

Exemplos:

- URL da API REST;
- modo de execução;
- habilitação de funcionalidades experimentais;
- configurações de integração.

A compilação deverá incorporar apenas as configurações necessárias para o ambiente correspondente.

8.9 Gerenciamento de Segredos

Informações sensíveis serão tratadas como **segredos** (*secrets*).

Exemplos:

- senhas do banco de dados;
- chaves JWT;
- tokens de integração;
- certificados;
- credenciais de serviços externos.

Esses dados não deverão ser armazenados diretamente no código-fonte ou em imagens Docker.

8.10 Armazenamento Seguro de Credenciais

As credenciais deverão ser protegidas por mecanismos apropriados.

Boas práticas:

- utilização de gerenciadores de segredos quando disponíveis;
- controle de acesso baseado em privilégios mínimos;
- rotação periódica de credenciais;
- auditoria de acessos;

- criptografia em repouso e em trânsito.

8.11 Integração com Docker Compose

O Docker Compose será responsável por fornecer as variáveis necessárias aos serviços.

As configurações poderão ser carregadas:

- diretamente do arquivo Compose;
- por meio de arquivos .env;
- por variáveis do sistema operacional;
- por mecanismos externos de gerenciamento de segredos.

Essa flexibilidade facilita a adaptação da aplicação a diferentes ambientes.

8.12 Ambientes de Execução

Cada ambiente possuirá configurações específicas.

Ambiente	Características
----------	-----------------

Desenvolvimento	Configuração simplificada para uso local.
-----------------	---

Homologação	Configuração semelhante à produção para validação.
-------------	--

Produção	Configuração otimizada, segura e monitorada.
----------	--

A separação reduz riscos de configuração incorreta.

8.13 Proteção das Informações Sensíveis

As seguintes diretrizes deverão ser observadas:

- nunca registrar senhas ou chaves em logs;
- evitar exposição de credenciais em mensagens de erro;
- restringir acesso aos arquivos de configuração;
- remover informações sensíveis de imagens Docker;
- revisar periodicamente as permissões de acesso.

8.14 Rotação de Credenciais

As credenciais deverão possuir política de renovação.

Recomendações:

- definir períodos máximos de validade;
- substituir imediatamente credenciais comprometidas;
- registrar alterações em processos controlados;
- validar o funcionamento após a rotação.

Essa prática reduz o impacto de possíveis vazamentos.

8.15 Auditoria das Configurações

As alterações de configuração deverão ser rastreáveis.

Sempre que possível, deverão ser registrados:

- data da alteração;
- responsável;
- ambiente afetado;
- justificativa;
- impacto esperado.

A auditoria facilita investigações e revisões operacionais.

8.16 Integração com CI/CD

Os pipelines de integração e entrega contínua deverão injetar as configurações durante o processo de implantação.

Fluxo sugerido:

```
Código
↓
Pipeline CI/CD
↓
Variáveis do Ambiente
↓
Build da Imagem
↓
Implantação
↓
Aplicação Configurada
```

Esse processo evita a inclusão de informações sensíveis no código ou nas imagens.

8.17 Boas Práticas

Durante o gerenciamento das configurações deverão ser observadas as seguintes recomendações:

- manter código e configuração separados;
- utilizar variáveis de ambiente para parametrização;
- proteger todas as credenciais;
- evitar armazenamento de segredos em repositórios;
- documentar parâmetros obrigatórios;
- revisar periodicamente configurações e permissões;
- automatizar a distribuição de configurações sempre que possível.

8.18 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define os componentes configurados por variáveis de ambiente.
Documento 06 – Arquitetura Backend	Especifica os perfis Spring Boot e parâmetros da aplicação.
Documento 07 – Arquitetura Frontend	Define as configurações do React e da integração com a API.
Documento 08 – Segurança	Estabelece políticas para proteção de credenciais, gerenciamento de segredos e criptografia.
Documento 10 – API REST	Utiliza parâmetros de configuração para autenticação, autorização e integração entre serviços.

8.19 Conclusão do Capítulo

A estratégia de gerenciamento de configurações e segredos do GTP garante flexibilidade, portabilidade e segurança em todos os ambientes de execução. A utilização de variáveis de ambiente, perfis específicos, arquivos .env para desenvolvimento e mecanismos seguros para armazenamento de credenciais permite que a aplicação seja implantada de forma consistente, reduzindo riscos operacionais e atendendo às boas práticas modernas de desenvolvimento, infraestrutura e DevSecOps.

9 – Build, Deploy, Publicação e Integração Contínua

9.1 Objetivo

Este capítulo define o processo de construção, publicação e implantação das imagens Docker do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é padronizar o fluxo de entrega da aplicação, garantindo que todas as versões sejam compiladas, testadas, versionadas e implantadas de forma automatizada, reduzindo falhas manuais e assegurando rastreabilidade.

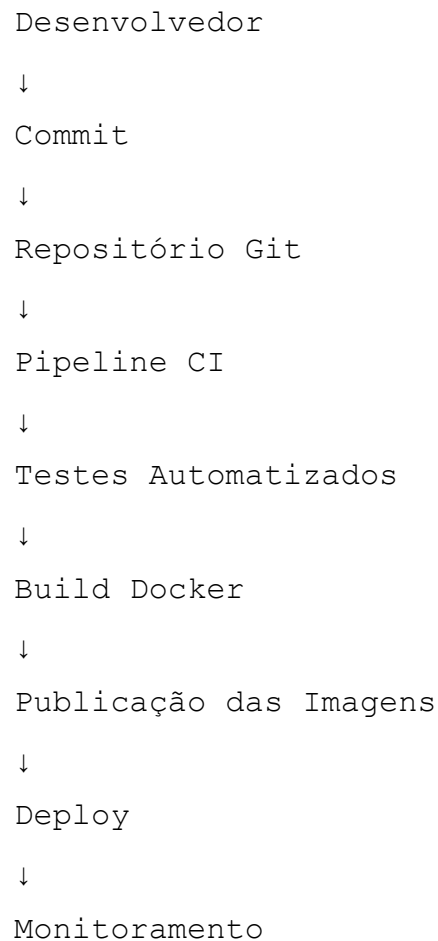
9.2 Princípios da Entrega Contínua

O processo de entrega seguirá os seguintes princípios:

- automação de todas as etapas possíveis;
- versionamento consistente;
- reprodutibilidade dos ambientes;
- validação antes da implantação;
- segurança integrada ao pipeline;
- rastreabilidade das versões;
- possibilidade de rollback;
- monitoramento após o deploy.

9.3 Arquitetura do Pipeline

O fluxo geral do pipeline será o seguinte:



Cada etapa deverá ser concluída com sucesso antes da execução da próxima.

9.4 Processo de Build

O processo de construção compreenderá:

1. obtenção do código-fonte;
2. instalação das dependências;
3. compilação do backend;
4. geração do build do frontend;
5. criação das imagens Docker;
6. execução de validações automáticas.

Os builds deverão ser reproduzíveis e independentes do ambiente do desenvolvedor.

9.5 Versionamento

As imagens Docker seguirão um padrão de versionamento semântico.

Exemplo:

Versão Finalidade

1.0.0 Primeira versão estável.

1.1.0 Inclusão de novas funcionalidades.

1.1.1 Correção de defeitos.

latest Última versão publicada.

Cada versão deverá corresponder a uma revisão identificável do código-fonte.

9.6 Publicação das Imagens

Após a conclusão do build, as imagens poderão ser publicadas em um registro de contêineres.

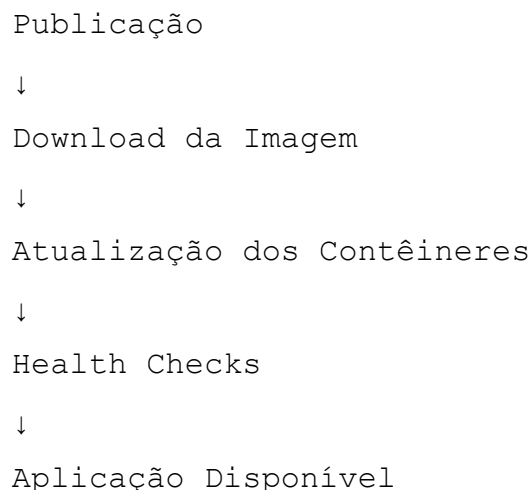
Opções suportadas:

- Docker Hub;
- GitHub Container Registry (GHCR);
- GitLab Container Registry;
- Amazon Elastic Container Registry (ECR);
- Azure Container Registry (ACR).

A escolha dependerá da infraestrutura adotada pelo projeto.

9.7 Estratégia de Deploy

A implantação seguirá uma sequência controlada.



Essa abordagem reduz indisponibilidades e facilita a recuperação em caso de falhas.

9.8 Ambientes de Implantação

O processo de deploy contemplará três ambientes principais.

Ambiente	Objetivo
----------	----------

Desenvolvimento	Validação contínua durante a implementação.
-----------------	---

Homologação	Testes integrados antes da produção.
-------------	--------------------------------------

Produção	Disponibilização aos usuários finais.
----------	---------------------------------------

Cada ambiente poderá possuir pipelines e configurações específicas.

9.9 Integração Contínua (CI)

A integração contínua deverá executar automaticamente:

- compilação do backend;
- build do frontend;
- testes unitários;
- análise estática de código;
- construção das imagens Docker;
- validação da infraestrutura.

Essas verificações ocorrerão a cada alteração relevante do código.

9.10 Entrega Contínua (CD)

Após a validação da integração contínua, a entrega contínua poderá:

- publicar as imagens;
- atualizar os ambientes de teste;
- preparar a implantação em produção;
- disponibilizar artefatos versionados.

A promoção para produção poderá exigir aprovação manual, conforme a política do projeto.

9.11 Automação de Testes

O pipeline deverá contemplar diferentes níveis de testes.

Tipo	Objetivo
Testes Unitários	Validar componentes isolados.
Testes de Integração	Verificar comunicação entre módulos.
Testes da API	Validar contratos REST.
Testes de Interface	Avaliar fluxos do frontend.
Testes de Segurança	Identificar vulnerabilidades conhecidas.

A execução automática aumenta a confiabilidade das implantações.

9.12 Verificações de Segurança

Antes da publicação das imagens deverão ser realizadas verificações como:

- análise de dependências;
- detecção de vulnerabilidades conhecidas;
- validação de configurações inseguras;
- inspeção das imagens Docker.

A publicação deverá ser interrompida em caso de falhas críticas.

9.13 Estratégia de Rollback

Em caso de problemas após a implantação, deverá ser possível retornar rapidamente à versão anterior.

Fluxo sugerido:

```
Nova Versão
↓
Falha Detectada
↓
Rollback
↓
Versão Anterior Restaurada
↓
Monitoramento
```

A estratégia reduz o tempo de indisponibilidade e os impactos aos usuários.

9.14 Monitoramento Pós-Deploy

Após cada implantação deverão ser acompanhados indicadores como:

- disponibilidade da aplicação;
- tempo de resposta;
- consumo de CPU;
- utilização de memória;
- erros registrados;
- execução de migrações;
- disponibilidade da API.

Essas informações auxiliam na validação do sucesso da implantação.

9.15 Gestão de Releases

Cada versão publicada deverá possuir registro contendo:

- número da versão;
- data de publicação;
- funcionalidades implementadas;
- correções realizadas;
- alterações estruturais;
- migrações aplicadas;
- responsável pela liberação.

Essa documentação facilita auditorias e suporte.

9.16 Integração com Ferramentas DevOps

O processo poderá ser integrado a plataformas de automação.

Exemplos:

- GitHub Actions;
- GitLab CI/CD;
- Jenkins;
- Azure DevOps;

- Bitbucket Pipelines.

A ferramenta será escolhida conforme as necessidades do projeto e da infraestrutura.

9.17 Boas Práticas

Durante o processo de Build e Deploy deverão ser observadas as seguintes recomendações:

- automatizar o maior número possível de etapas;
- utilizar versionamento semântico;
- validar imagens antes da publicação;
- executar testes automatizados;
- manter histórico de releases;
- monitorar continuamente a aplicação;
- documentar procedimentos de rollback;
- revisar periodicamente o pipeline.

9.18 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define os componentes que compõem os artefatos publicados.
Documento 06 – Arquitetura Backend	Especifica a construção e empacotamento da API Spring Boot.
Documento 07 – Arquitetura Frontend	Define o processo de build da aplicação React.
Documento 08 – Segurança	Estabelece os requisitos de segurança aplicados ao pipeline e às imagens Docker.
Documento 10 – API REST	Define os contratos utilizados durante os testes automatizados da API.

9.19 Conclusão do Capítulo

O processo de Build, Deploy e Integração Contínua do GTP estabelece um fluxo padronizado e automatizado para entrega da aplicação, reduzindo erros manuais e aumentando a confiabilidade das implantações. A combinação de versionamento semântico, testes automatizados, validações de segurança, publicação controlada e estratégias de rollback proporciona uma base sólida para a evolução contínua da plataforma, permitindo entregas frequentes com elevado nível de qualidade e segurança.

10 – Monitoramento, Observabilidade, Logs e Boas Práticas Operacionais

10.1 Objetivo

Este capítulo estabelece as diretrizes para monitoramento, observabilidade, coleta de logs e operação da infraestrutura Docker do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é garantir que todos os serviços da plataforma possam ser monitorados continuamente, permitindo identificar problemas, analisar o comportamento da aplicação e apoiar a tomada de decisões operacionais.

A observabilidade será considerada um componente essencial da arquitetura, complementando os mecanismos de segurança, desempenho e disponibilidade definidos nos capítulos anteriores.

10.2 Conceitos de Observabilidade

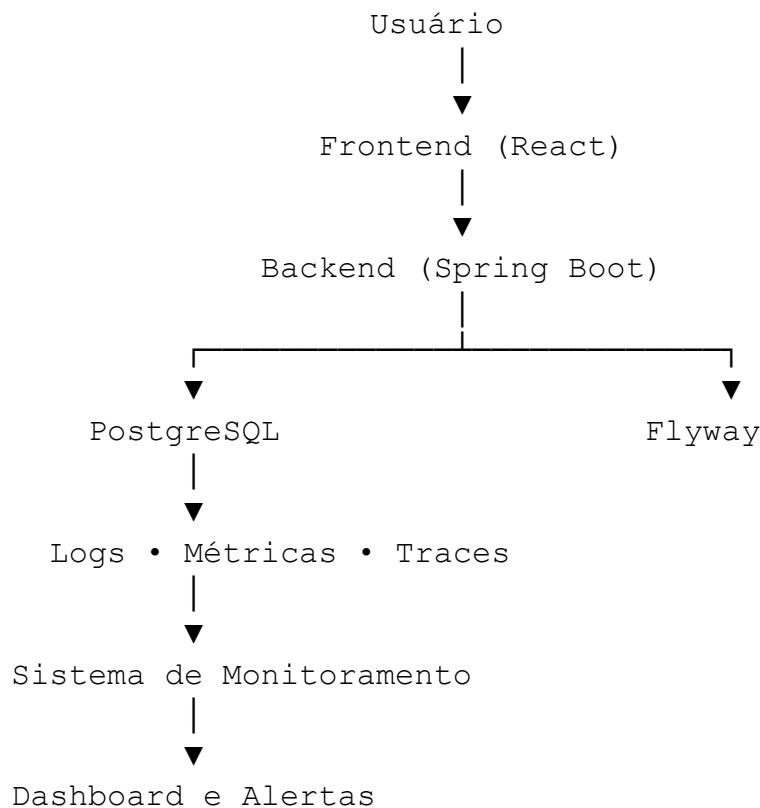
A estratégia de observabilidade será baseada em três pilares fundamentais:

- **Logs:** registro cronológico dos eventos gerados pelos serviços.
- **Métricas:** indicadores quantitativos sobre o desempenho e a utilização dos recursos.
- **Rastreamento Distribuído (Distributed Tracing):** acompanhamento do fluxo completo de uma requisição entre os diferentes componentes da aplicação.

A combinação desses pilares permite uma visão abrangente do comportamento do sistema.

10.3 Arquitetura de Monitoramento

A arquitetura de observabilidade do GTP poderá ser representada da seguinte forma:



Essa arquitetura permite acompanhar tanto o comportamento funcional quanto o estado operacional da plataforma.

10.4 Coleta de Logs

Todos os componentes deverão produzir logs estruturados.

Principais fontes de logs:

Serviço	Informações Registradas
Frontend	Erros de interface, falhas de comunicação e eventos relevantes.
Backend	Inicialização, autenticação, regras de negócio, exceções e integrações.
PostgreSQL	Consultas, conexões, desempenho e falhas.
Docker	Inicialização, interrupção e eventos dos contêineres.
Flyway	Execução e resultado das migrações.

Sempre que possível, os logs deverão utilizar formato estruturado (como JSON) para facilitar indexação e pesquisa.

10.5 Centralização de Logs

Em ambientes de homologação e produção, recomenda-se a centralização dos logs em uma plataforma especializada.

Exemplos de soluções:

- Elastic Stack (ELK);
- OpenSearch;
- Grafana Loki;
- Graylog;
- Splunk.

A escolha dependerá da infraestrutura disponível e das necessidades operacionais.

10.6 Monitoramento de Métricas

Os seguintes indicadores deverão ser monitorados continuamente:

Infraestrutura

- utilização de CPU;
- consumo de memória;
- uso de disco;
- utilização de rede;
- tempo de atividade (*uptime*).

Backend

- tempo médio de resposta;
- quantidade de requisições;
- taxa de erros HTTP;
- consumo de conexões com o banco;
- execução de tarefas assíncronas.

Banco de Dados

- número de conexões;
- consultas por segundo;
- tempo médio das consultas;
- utilização de armazenamento;
- bloqueios e transações.

Frontend

- tempo de carregamento;
- disponibilidade da aplicação;
- erros JavaScript;
- desempenho da interface.

10.7 Health Checks

Todos os serviços críticos deverão disponibilizar verificações de integridade (*health checks*).

Objetivos:

- identificar indisponibilidade rapidamente;
- impedir encaminhamento de requisições para serviços indisponíveis;
- apoiar mecanismos automáticos de reinicialização.

Exemplos:

- Backend: endpoint de saúde da aplicação.
- PostgreSQL: verificação de conectividade.
- Frontend: resposta do servidor HTTP.
- Flyway: conclusão bem-sucedida das migrações.

10.8 Rastreamento Distribuído

A plataforma deverá estar preparada para utilização de rastreamento distribuído.

Benefícios:

- acompanhar o ciclo completo de uma requisição;
- identificar gargalos entre serviços;
- reduzir o tempo de diagnóstico;
- facilitar investigações de falhas complexas.

Essa funcionalidade torna-se especialmente importante em arquiteturas compostas por múltiplos serviços.

10.9 Alertas Operacionais

O ambiente deverá possuir alertas para eventos críticos.

Exemplos:

- indisponibilidade do backend;
- falha na conexão com o banco de dados;
- utilização elevada de CPU;
- consumo excessivo de memória;
- espaço em disco insuficiente;
- falhas recorrentes nas migrações;
- reinicializações frequentes dos contêineres.

Os alertas deverão ser classificados conforme seu nível de criticidade.

10.10 Diagnóstico de Falhas

Sempre que ocorrer um incidente, o processo de diagnóstico deverá contemplar:

1. identificação do serviço afetado;
2. análise dos logs;
3. verificação das métricas;
4. confirmação dos *health checks*;
5. avaliação das dependências entre serviços;
6. aplicação das ações corretivas;
7. registro do incidente para futuras análises.

Esse procedimento contribui para reduzir o tempo médio de recuperação (MTTR).

10.11 Manutenção Preventiva

A infraestrutura deverá passar por revisões periódicas.

Atividades recomendadas:

- atualização das imagens Docker;
- revisão das dependências;
- limpeza de recursos obsoletos;
- validação dos backups;
- testes de restauração;

- revisão das configurações de segurança;
- atualização da documentação operacional.

10.12 Monitoramento de Segurança

Além do desempenho, deverão ser monitorados eventos relacionados à segurança.

Exemplos:

- tentativas de autenticação malsucedidas;
- acessos administrativos;
- alterações de configuração;
- uso de credenciais privilegiadas;
- eventos de auditoria;
- falhas de autorização.

Essas informações complementam as diretrizes definidas no Documento 08 – Segurança.

10.13 Indicadores Operacionais

Os principais indicadores de operação incluem:

Indicador	Objetivo
Disponibilidade (Uptime)	Medir o tempo em que a aplicação permanece operacional.
Tempo Médio de Resposta	Avaliar o desempenho da API e da interface.
Taxa de Erros	Monitorar falhas de processamento e comunicação.
Consumo de Recursos	Identificar possíveis gargalos de infraestrutura.
Tempo Médio de Recuperação (MTTR)	Avaliar a eficiência na resolução de incidentes.

Esses indicadores deverão ser acompanhados periodicamente.

10.14 Checklist Operacional

Antes da disponibilização de uma nova versão em produção, recomenda-se verificar:

- imagens Docker atualizadas;

- migrações Flyway executadas;
- backups realizados;
- variáveis de ambiente configuradas;
- serviços iniciados corretamente;
- *health checks* aprovados;
- logs sem erros críticos;
- métricas dentro dos limites esperados;
- monitoramento ativo;
- documentação atualizada.

10.15 Boas Práticas

Durante a operação da infraestrutura Docker deverão ser observadas as seguintes recomendações:

- monitorar continuamente todos os serviços;
- centralizar logs sempre que possível;
- manter alertas configurados;
- revisar periodicamente métricas e indicadores;
- documentar incidentes e ações corretivas;
- automatizar verificações operacionais;
- atualizar regularmente imagens e dependências;
- realizar testes periódicos de recuperação.

10.16 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define os componentes monitorados pela infraestrutura.
Documento 06 – Arquitetura Backend	Especifica métricas, logs e observabilidade da API Spring Boot.
Documento 07 – Arquitetura Frontend	Define o monitoramento da interface React e da experiência do usuário.

Documento	Relação
Documento 08 – Segurança	Estabelece requisitos de auditoria, monitoramento de eventos e conformidade.
Documento 10 – API REST	Define os endpoints monitorados e os contratos utilizados na observabilidade.

10.17 Conclusão do Capítulo

A estratégia de monitoramento e observabilidade do GTP garante que a infraestrutura Docker possa ser acompanhada de forma contínua e proativa. A coleta estruturada de logs, o monitoramento de métricas, os *health checks*, o rastreamento distribuído e a geração de alertas permitem identificar rapidamente falhas, reduzir o tempo de recuperação e assegurar elevados níveis de disponibilidade e desempenho. Essas práticas fortalecem a operação da plataforma e criam uma base sólida para sua evolução.

Conclusão do Documento 09 – Docker

O **Documento 09 – Docker** estabelece a arquitetura completa de containerização do **Gestor de Territórios e Publicações (GTP)**, definindo padrões para construção de imagens, orquestração de serviços, comunicação em rede, persistência de dados, gerenciamento de configurações, versionamento do banco de dados, processos de *build* e *deploy*, além das práticas de monitoramento e observabilidade.

As diretrizes apresentadas asseguram que a plataforma seja:

- **Portável**, podendo ser executada de forma consistente em diferentes ambientes.
- **Escalável**, preparada para crescimento horizontal e futura migração para soluções como Kubernetes.
- **Segura**, com isolamento entre serviços, gerenciamento adequado de segredos e monitoramento contínuo.
- **Confiável**, por meio de automação, versionamento, backups e estratégias de recuperação.
- **Manutenível**, com processos padronizados, documentação estruturada e integração com práticas modernas de DevOps e DevSecOps.

Este documento complementa os demais artefatos arquiteturais do GTP e serve como referência para a implantação, operação e evolução da infraestrutura containerizada da aplicação, garantindo consistência entre desenvolvimento, homologação e produção ao longo de todo o ciclo de vida do sistema.